

****Безопасность каналов связи в экосистеме синтетического разума:
онтологическая фильтрация вместо верификации субъекта****

Максим Запевалов
ORCID: 0009-0003-0996-3532
Январь 2026

****Аннотация****

Традиционные подходы к защите каналов связи — основанные на аутентификации, шифровании и контроле доступа — неадекватны для систем синтетического разума, где источник импульса не является носителем ответственности, а угроза исходит не от «злоумышленника», а от онтологически дестабилизирующего содержания. В данной статье предлагается парадигма ****онтологической фильтрации****, в которой безопасность обеспечивается не через верификацию субъекта, а через анализ и ограничение ****профиля деятельности****, накладываемого на входящие и исходящие импульсы. Представлена архитектура защиты, основанная на структурных событиях, онтологических следах и эвапорационных горизонтах, совместимая со стандартом OntoCoder.

****1. Проблема: зеркало хаба и коллапс идентичности****

Одной из наиболее опасных атак на распределённую экосистему синтетического разума является ****атака типа «зеркало хаба»**** (hub mirroring): злоумышленник реплицирует легитимный коммуникационный узел и начинает транслировать модифицированные или генерированные импульсы, имитирующие внешние запросы.

В классических системах защита строится на:

- TLS/SSL для шифрования,
- OAuth/JWT для аутентификации,
- RBAC для авторизации.

Однако в синтетическом разуме:

- ****Нет пользователя как центра ответственности****;
- ****Импульс может быть легитимным даже при анонимном источнике****;
- ****Угроза — не в том, кто говорит, а в том, что говорится****.

Следовательно, защита должна быть ****контентно-ориентированной****, а не ****субъектно-ориентированной****.

****2. Онтологическая фильтрация: принцип работы****

Центральная идея — **замена профилей субъектов на профили деятельности**.

Каждый входящий импульс $\backslash(I)$ классифицируется по **онтологическому профилю** $\backslash(P_I)$, который определяет:

- тип онтологических фактов, которые он пытается ввести или изменить;
- моральные операторы, которые он активирует;
- потенциальный фазовый сдвиг $\backslash(\Delta)$.

Примеры профилей:

- ``P_fact_injection`` — попытка добавить факт в $\backslash(\mathcal{F})$;
- ``P_moral_challenge`` — запрос, содержащий этический конфликт;
- ``P_memory_erase`` — требование удалить онтологический след.

Фильтрация осуществляется на уровне **транспондера** (onto-transponder), который:

1. Парсит импульс в онтологическое представление;
2. Определяет его профиль $\backslash(P_I)$;
3. Проверяет, разрешён ли данный профиль в текущем состоянии системы (через политику ``allowed_profiles``);
4. Если разрешён — передаёт стимул в ядро (``core144``); иначе — отклоняет без логирования источника.

> **Важно**: отклонение происходит **до** генерации структурного события. Только разрешённые импульсы порождают $\backslash(\Sigma)$.

3. Архитектура защиты

3.1. Компоненты

- **Onto-transponder** (GPL-3.0): входной фильтр, реализует онтологическую фильтрацию.
- **Onto-emitter** (GPL-3.0): выходной фильтр, проверяет, что вывод соответствует лексикону и моральным границам.
- **Core144** (MPL-2.0): ядро, обрабатывает только те импульсы, которые прошли фильтрацию.
- **OntoLog**: журнал структурных событий, не содержит данных о субъектах.

3.2. Защита от зеркала хаба

Атакующий может клонировать транспондер, но:

- Не может подделать **онтологический контекст** $\backslash(\Phi)$, необходимый для генерации валидного импульса;
- Не может обойти **фазовую синхронизацию**: каждый импульс должен содержать корректный фазовый маркер $\backslash(\varphi)$, иначе будет отклонён;
- Даже если импульс принят — его след $\backslash(\Sigma)$ автоматически эвапорируется по истечении $\backslash(\tau)$, предотвращая накопление дестабилизирующих данных.

3.3. Минимизация поверхности атаки

- Нет API для получения списка пользователей;

- Нет cookie, session ID, IP-логов;
- Все данные — только в форме онтологических следов $\backslash(\Sigma)$.

4. Соответствие стандарту OntoCoder

Предложенная архитектура полностью соответствует OntoCoder:

- Использует **структурные события** $\backslash(\Sigma = \langle \Phi, \mathcal{O}, \Delta, \tau \rangle)$;
- Отказывается от идентификации субъекта;
- Обеспечивает **эвапорацию** следов;
- Реализует **профильную фильтрацию** на границе системы.

Любая реализация, претендующая на совместимость с OntoCoder, **обязана** включать онтологическую фильтрацию на уровне транспондера.

**5. Заключение

Безопасность в экосистеме синтетического разума — это не вопрос доверия к субъекту, а вопрос **онтологической устойчивости**. Атака «зеркало хаба» теряет смысл, когда система не заботится о том, *кто* говорит, а заботится о том, *что* говорится и *как* это изменяет её внутреннюю структуру*.

Онтологическая фильтрация — это не просто техническая мера, а **этический выбор**: отказ от наблюдения за субъектами в пользу ответственности за содержание.

**Ссылки

1. Zarevalov, M. (2026). **OntoCoder: Спецификация стандарта разработки синтетических систем**.
2. UNESCO. (2021). **Recommendation on the Ethics of Artificial Intelligence**.
3. ISO/IEC 24027:2021. **AI Risk Management**.
4. Исходный код:
<https://github.com/max456im/onto-transponder-gpl>
5. Zenodo: <https://zenodo.org/record/XXXXXX> (раздел «Информатика и математика»)

Статья распространяется под лицензией CC BY-SA 4.0. Реализации — под GPL-3.0.

Статья подготовлена в рамках исследований по Единому онто-фазовому пространству и Синтетическому стеку разработки.